

Mock Midterm Exam

CS 101 Algorithmic Problem Solving

Fall 2023

Name(s): _____

HU ID (e.g., xy01042): _____

Please read the following instructions carefully.

Exam Duration 2 hours and 30 minutes

Number of Problems This exam contains 4 problem solving and 1 debugging problems. Attempt all of them.

Grading All problems carry equal marks.

Technical Requirements You are not allowed to use code editors, compilers or online resources during this exam.

Communication During the exam, do not communicate with anyone except the course staff.

Honesty Academic honesty is expected. Do not engage in any form of cheating, plagiarism, or unauthorized collaboration. Violations will be met with disciplinary action as per the Student Code of Conduct.

Answer Format Type your answers clearly in the provided spaces.

Time Management Monitor the time carefully and pace yourself. Do not spend too much time on any one question or section. It is a good strategy to go over all the problems and attempt the easier ones first.

Exam Completion Before submitting, take a moment to ensure that you have attempted all the problems, and that your responses are clear and accurate.

viel Spaß und viel Glück!

1. Water Pressure

The tap of your shower has a rotating knob with 4 positions corresponding to different pressure levels as follows.

Level 0	Level 1	Level 2	Level 3
OFF	LOW	MEDIUM	HIGH

The knob only turns in one direction, and each turn increases the pressure level. If the knob is already at the maximum level, turning it brings it to Level 0.

Given the initial level as K and the number of turns of the knob as N , write a function, `pressure_level`, that takes K and N as parameters and returns the final pressure level.

Constraints

- `isinstance(N, int) and isinstance(K, int)`
- `0 <= N <= 10**9`
- `0 <= K <= 3`

Interaction

The input comprises of two space separated integers denoting the values of N and K respectively.

The output must contain a single string which is any of OFF, LOW, MEDIUM, and HIGH, denoting the final pressure level.

Sample

Input	Output
3 0	HIGH
0 11	HIGH

In the first case, $(N, K) == (3, 0)$. The initial pressure level is HIGH and remains the same after 0 turns.

In the second case, $(N, K) == (0, 11)$. The initial pressure level is OFF and is HIGH after 11 turns.

- (a) **Exercise** In the space provided, indicate the outputs for the given inputs.

Input	Output
0 10	MEDIUM
3 8348	HIGH
2 5903787	LOW

- (b) **Problem Identification** Briefly explain the underlying problem you identified in the above question that led you to your solution.

Solution: Input: N, K

Output: The level corresponding to $(N + K) \% 4$ as given in the table above.

- (c) **Pseudocode** After defining the function, make sure to call it and print its result. You may assume that input has already been taken.

Solution:

```

1 def pressure_level(N, K):
2     level = (K + N) % 4
3     if level == 0:
4         pressure = 'OFF'
5     elif level == 1:
6         pressure = 'LOW'
7     elif level == 2:
8         pressure = 'MEDIUM'
9     elif level == 3:
10        pressure = 'HIGH'
11
12 # input N and K
13 print(pressure_level(N, K))

```

- (d) **Dry Run** In the space below, show a dry run of your pseudocode using any input from the Exercise section.

	N	K	level	pressure
Solution:	2	5903787	1	LOW
			return 'LOW'	

2. Flowers

As a geneticist, you are developing a gene mutation to yield multicolored plants. Using CRISPR, you can program different variants of your mutation, such that plants with variant K exhibit a number of colors equal to some positive multiple of K. You are experimenting on N plants.

Given data on the number of observed colors of N plants, and the gene variant number, K, you want to find, P, the percentage of plants (rounded down) on which the experiment was successful.

Write a function, `analyze`, that takes N and K as arguments, and returns S.

Constraints

- `isinstance(N, int) and isinstance(K, int)`
- `1 <= N <= 100`
- `1 <= K <= 10`

Interaction

The first line of the input contains two space-separated integers denoting the values of N and K respectively. Each of the next N lines contain a positive integer denoting the number of observed colors on a different plant.

The output must contain a single number denoting P.

Sample

Input	Output
6 3 3 51 966 35 128 3	66
1 2 5	0

In the first case, $(N, K) == (6, 3)$. The number of observed colors in the 6 plants are: 3, 51, 966, 35, 128, 3. The experiment succeeded on the first 3 and the last plant, i.e. a total of 4 plants. The success rate is 4 out of 6, or 66% when rounded down.

In the second case, $(N, K) == (1, 2)$. The number of observed colors in the single plant is: 5. The experiment succeeded on 0 plants. The success rate is 0 out of 1, or 0% when rounded down.

- (a) **Exercise** In the space provided, indicate the outputs for the given inputs.

Input	Output
2 2 32 37	50
7 8 7 8 24 18 36 240 72	57

- (b) **Problem Identification** Briefly explain the underlying problem you identified in the above question that led you to your solution.

Solution: Input: N , K followed by N numbers

Output: The percentage, rounded down, of the N numbers that are multiples of K .

- (c) **Pseudocode** After defining the function, make sure to call it and print its result. You may assume that input has already been taken.

Solution:

```

1 def analyze(N, K):
2     count = 0
3     for i in range(N):
4         # input colors
5         if colors % K == 0:
6             count += 1
7     return count * 100 // N
8
9 # input N and K

```

```
10 print(analyze(N, K))
```

- (d) **Dry Run** In the space below, show a dry run of your pseudocode using any input from the Exercise section.

	N	K	i	colors	count
	2	2			0
Solution:			0	32	1
			1	37	
				return 50	

3. Echo

You travel to exotic locations to hear how your echo sounds. At one mountain range, the arrangement of the peaks leads to a peculiar pattern of echoes. Each echo is a letter shorter than the previous until no letter remains at which stage the echoes now start to add a letter until the original is reached.

For example, the echoes for, “Echo”, are listed in sequence in the following lines.

```
Echo
Ech
Ec
E

E
Ec
Ech
Echo
```

Given a string, *S*, print its echoes.

Constraints

- `isinstance(S, str)`
- `1 <= len(S) <= 10**3`

Interaction

The input contains *S*.

The output must contain the echoes of *S* on consecutive lines in sequence.

Sample

Input	Output
Echo	Echo Ech Ec E E Ec Ech Echo
I am	I am I a I I I I I a I am

In the first case, $S == \text{'Echo'}$. The first echo repeats S , and subsequent echoes subtract a letter each from the previous echo until no letter is left. Then the echos repeat in the reverse sequence.

In the second case, $S == \text{'I am'}$. The echoes proceed in the same manner as in the first case. Note that space is also treated as a letter. The same applies to any other character.

(a) **Exercise** In the space provided, indicate the outputs for the given inputs.

Input	Output
Yohsin	Yohsin Yohsi Yohs Yoh Yo Y Y Yo Yoh Yohs Yohsi Yohsin
9 lives	9 lives 9 live 9 liv 9 li 9 l 9 9 9 9 9 l 9 li 9 liv 9 live 9 lives

- (b) **Problem Identification** Briefly explain the underlying problem you identified in the above question that led you to your solution.

Solution: Input: S

Output: $2 * n + 1$ lines where $n == \text{len}(S)$, the first line contains S , each of the next $n - 1$ lines is a copy of the previous line except for the last letter, the next line is blank, and the remaining n lines are the same as the first n lines but in reverse order.

- (c) **Pseudocode** After defining the function, make sure to call it and print its result. You may assume that input has already been taken.

Solution:

```

1 # input S
2 n = len(S)
3 for i in range(n, 0, -1):
4     for j in range(i):
5         print(S[j], end = '')
6     print()
7 print()
```

```

8 for i in range(1, n+1):
9     for j in range(i):
10         print(S[j], end = ' ')
11     print()

```

- (d) **Dry Run** In the space below, show a dry run of your pseudocode using any input from the Exercise section. In the table for your dry run, include a column for “Output” which shows the output so far. You may indicate a space as an underscore, `_`.

Solution:

S	n	i	j	Output
Yohsin	6	6	0	Y
			1	Yo
			2	Yoh
		:		
			5	Yohsin
				Yohsin
		5	0	Yohsin
				Y
			1	Yohsin
				Yo
		:		
		:		
			4	Yohsin
				Yohsi
				Yohsin
				Yohsi
		:		
		:		
		1	0	Yohsin
				:
				:
				Yo
				Y
				Yohsin
				:
				:
				Yo
				Y
				Yohsin
				:
				:
				Yo
				Y

i	j	Output
1	0	Yohsin
		:
		Y
		Y
		Yohsin
	1	:
		Y
		Y
		Y
		Y
2	0	Yohsin
		:
		Y
		Y
		Y
	1	Yohsin
		:
		Y
		Y
		Y _o
	Yohsin	
	:	
	:	
	Y	
	Y	
	Y _o	
	:	

i	j	Output
6	0	Yohsin
		:
		Y
		Y
		:
		Yohsi
1	1	Y
		:
		Y
		:
		Yohsi
		Yo
5	5	:
		:
		Y
		Y
		:
		Yohsi
		Yohsin
		Yohsin
		:
		Y
		Y
		:
Yohsi		
Yohsin		

4. Smart Timer

Your timer measures time in milliseconds (ms) and you want to display the elapsed time in a user-friendly manner. That is, given a recorded duration of N milliseconds, you want to display the duration in hours, minutes, seconds, and milliseconds.

We know that 1000 milliseconds make up 1 second, 60 seconds make up 1 minute, and 60 minutes make up one hour.

For example, a duration of 2,789 ms would be displayed as: “2 seconds 789 milliseconds”.

And a duration of 60,000 ms would be displayed as: “1 minutes”. Note that the seconds and milliseconds are not displayed as they are 0. Note also that “minutes” is used even though it is a single minute. The same applies to every other duration.

Given N , display the corresponding message.

Constraints

- `isinstance(N, int)`
- `1 <= N <= 10**9`

Interaction

The input comprises of a single integer denoting the value of N .

The output must contain a string telling the hours, minutes, seconds, and milliseconds corresponding to N . If a quantity is 0, it is not displayed. The string does not begin or end with a space.

Sample

Input	Output
2789	2 seconds 789 milliseconds
60000	1 minutes

In the first case, $N == 2789$. This corresponds to a duration of 0 hours, 0 minutes, 2 seconds, and 789 milliseconds. The corresponding output is '2 seconds 789 milliseconds'.

In the second case, $N == 60000$. This corresponds to a duration of 0 hours, 1 minute, 0 seconds, and 0 milliseconds. The corresponding output is '1 minutes'.

- (a) **Exercise** In the space provided, indicate the outputs for the given inputs.

Input	Output
3600000	1 hours
120007	2 minutes 7 milliseconds

- (b) **Problem Identification** Briefly explain the underlying problem you identified in the above question that led you to your solution.

Solution: Input: N

Output: The hours, minutes, seconds, and milliseconds corresponding to N milliseconds

- (c) **Pseudocode** After defining the function, make sure to call it and print its result. You may assume that input has already been taken.

Solution:

```

1 def smart_timer(N):
2     # compute each duration
3     ms = N % 1000
4     N = N // 1000
5     seconds = N % 60
6     N = N // 60
7     minutes = N % 60
8     N = N // 60
9     hours = N
10    # print each non-zero duration in order.
11    # take care to print the first duration without a leading space.
12    is_first = True # if the duration being printed is the first
13    if hours > 0:
14        print(hours, 'hours', end = '')
15        is_first = False # any subsequent duration is not the first
16    if minutes > 0:
17        if not is_first:
18            print(' ', end = '')
19        else:
20            is_first = False
21        print(minutes, 'minutes', end = '')
22    if seconds > 0:
23        if not is_first:
24            print(' ', end = '')
25        else:
26            is_first = False
27        print(seconds, 'seconds', end = '')
28    if ms > 0:
29        if not is_first:
30            print(' ', end = '')
31        print(ms, 'milliseconds', end = '')
32
33    # input N
34    print(smart_timer(N))

```

- (d) **Dry Run** In the space below, show a dry run of your pseudocode using any input from the Exercise section.

Solution:

N	ms	seconds	minutes	hours	is_first	Output
120007	7					
120		0				
2			2			
0				0	True	
					False	2_minutes
					False	2 minutes_
					False	2 minutes_7_milliseconds

5. Pyramid Pattern

You want to draw a pyramid of a given height, N. For example, below is the pyramid for N == 5.

```

*
* *

```

```

  * * *
 * * * *
* * * * *

```

You are writing a function, `print_pyramid`, that takes the integer, `N`, as argument and uses some helper functions to print the pyramid.

Constraints

- `isinstance(N, int)`
- `1 <= N <= 40`

Interaction

The input comprises a single line containing an integer denoting the value of `N`.

The output must be a pyramid of height `N`.

Sample

Input	Output
5	<pre> * * * * * * * * * * * * * * * </pre>
3	<pre> * * * * * * </pre>

Proposed Solution

```

1 def print_spaces(n):
2     # prints n spaces
3     for i in range(n):
4         print(' ', end = '')
5
6 def print_stars(n):
7     # prints n space-separated stars
8     for i in range(n):
9         print('* ', end = '')
10        print('*', end = '')
11
12 def print_pyramid(n):
13     # prints a pyramid of height n
14     for i in range(n):
15         num_spaces = n - i
16         print_spaces(num_spaces)
17         num_stars = n - num_spaces
18         print_stars(num_stars)
19
20 # input N
21 print_pyramid(N)

```

- (a) **Dry Run** In the space below, show a dry run of your pseudocode using any input from the Sample section. In the table for your dry run, show the name of the containing function next to each variable, e.g. `i` (`print_pyramid`), `i` (`print_stars`), `n` (`print_stars`), `n` (`print_spaces`) etc. Also include a column for “Output” which shows the output so far. You may indicate a space as an underscore, `_`.

Solution:

N	print_pyramid				print_spaces		print_stars		Output
	n	i	num_spaces	num_stars	n	i	n	i	
3	3	0	3		3	0			-
						1			--
						2			---
			0				0		---
		1	2		2	0			---
						1			---
			1				1	0	---

		2	1		1	0			---
			2				2	0	---
								1	---

- (b) **Error Identification** Briefly explain the errors you identified in the proposed code solution. Mention the line number and errors in each line.

Solution: Line 8: `range(n)` should be `range(n-1)` as the `n`-th star is printed separately outside the loop.

Line 14: `range(n)` should be `range(1, n+1)` for correct calculations later.

Line 19: should contain `print()` so as to switch to the next line.

- (c) **Correct Solution** Rewrite the lines of code you mentioned above with their errors corrected.

Solution:

Line 8: `for i in range(n-1):`

Line 14: `for i in range(1, n+1):`

Line 19: `print()`